

1-TP2

Dupla: Arthur Bicalho e Fernando Leite Pagani

(i) Código de submissão

```
#include <stdio.h>

#define MOD 998244353LL

long long gauss(long long n){
    if(n%2 == 0)
        return(n/2 % MOD) * ((n + 1) % MOD) % MOD;
    else
        return (n % MOD) * ((n + 1) / 2 % MOD) % MOD;
}

int main(void){
    long long A, B, C;
    scanf("%lld%lld%lld", &A, &B, &C);
    long long resultado = gauss(A) * gauss(B) % MOD * gauss(C) % MOD;

    printf("%lld\n", resultado);
    return 0;
}
```

(ii) Screenshot de aceite

Source Code [\[Collapse\]](#)

```
1 #include <stdio.h>
2
3 #define MOD 998244353LL
4
5 long long gauss(long long n){
6     if(n%2 == 0)
7         return(n/2 % MOD) * ((n + 1) % MOD) % MOD;
8     else
9         return (n % MOD) * ((n + 1) / 2 % MOD) % MOD;
10 }
11
12 int main(void){
13     long long A, B, C;
14     scanf("%lld%lld%lld", &A, &B, &C);
15     long long resultado = gauss(A) * gauss(B) % MOD * gauss(C) % MOD;
16     printf("%lld\n", resultado);
17     return 0;
18 }
19
20
```

Submission Info

Submission Time	2026-06-07 13:06:16
Task	A - Simple Math
User	Nietzsche_bica 
Language	C23 (GCC 14.2.0)
Score	300
Code Size	426 Byte
Status	AC
Exec Time	1 ms
Memory	1640 KiB

(iii) Manuscrito

A - Simple Math

• Demonstração Matemática

$$S = \sum_{a=1}^A \sum_{b=1}^B \sum_{c=1}^C a \cdot b \cdot c$$

$$S = \sum_{a=1}^A a \cdot \sum_{b=1}^B b \cdot \sum_{c=1}^C c$$

Soma de Gauss: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

$$S = \frac{A(A+1)}{2} \cdot \frac{B(B+1)}{2} \cdot \frac{C(C+1)}{2}$$

• Justificativa da Implementação no Código

Se não utilizar o inverso modular, o resultado fica quebrado, por causa da divisão por 2 após aplicar o operador de resto.

Se n for par ($n \% 2 == 0$): O número n é divisível por 2 de forma exata.

Código faz $(n/2) * (n+1)$.

Se n for ímpar: Então, obrigatoriamente, $(n+1)$ será par e divisível por 2 de forma exata. O código faz $n * ((n+1)/2)$.

• Uso do operador Módulo

Para evitar que a multiplicação $(A * B * C)$ estoure o limite de memória, aplica-se $\% \text{MOD}$ após cada operação, mantendo o valor sempre controlado e dentro do scope.

Em mais, long long resultado = gauss(A) * gauss(B) $\% \text{MOD}$ * gauss(C) $\% \text{MOD}$, executando o produto das três somas simplificadas.

Resolvendo o problema com apenas 1 linha de cálculo direto.

$$2 \cdot 01 \cdot 82 = 82 \cdot 2 \cdot 01$$

$$12 = 8 * 8 \cdot 2 \cdot 01$$

Análise de Complexidade de Tempo

função gauss: a fórmula da Soma de Gauss, realiza apenas uma verificação condicional (if) e um número fixo de operações matemáticas básicas. Nenhum depende do valor de n .

função main: Executa operações de entrada, chama a função GAUSS exatamente três vezes (para A, B e C), realiza mais algumas multiplicações e módulos, e finaliza com saída de dados.

Classe de complexidade: como o número de instruções executadas é o mesmo independentemente do tamanho dos variáveis de entrada, temos $O(1)$. O que vale também para a complexidade de espaço.

Cálculo do Tempo de Execução

$T(n) =$ Número total de operações no código

Operações por segundo do processador

K : quantidade total de operações

~~3 instruções/leitura (scanf)~~ ~~5 operações aritméticas dentro de cada chamada de gauss (total de 15)~~

~~4 operações aritméticas na main~~ ~~1 operação de saída (printf)~~

gauss: Total por chamada 8 operações

$n/2 == 0$: 1 operação de módulo, 1 comparação lógica (2 operações)

$n/2 < (ou > n+1)$: 1 adição e 1 divisão (2 operações)

1 MOD: 2 operações de módulo (2 operações)

Multiplicação central: (1 operação)

1 MOD final: 1 operação de módulo (1 operação)

main():

3 chamadas da função gauss: $3 \times 8 = 24$ operações

gauss(A) * gauss(B): 1 operação

1 MOD: 1 módulo (1 operação)

* gauss(C): 1 operação

% MOD: 1 operação

Total: 28 operações

$K=28$

$$T = \frac{K}{10^8} = \frac{28}{10^8} = 2,8 \cdot 10^{-7} s$$